

Fine-Tuning a Local LLM into a MITRE ATT&CK Specialist

End-to-End Technical Report — CyberCase Intelligence Framework

CyberCase Intelligence Framework

June 2026

Contents

1. Executive Summary	2
2. Objectives and Constraints	2
2.1 Objectives	2
2.2 Hardware constraint (the key design driver)	3
3. Base Model	3
4. Fine-Tuning Method	3
4.1 LoRA (Low-Rank Adaptation)	3
4.2 QLoRA (Quantised LoRA)	3
4.3 LoRA configuration	3
4.4 Response-only loss masking	4
5. Tooling / Frameworks	4
6. Dataset	4
6.1 Source	4
6.2 Construction pipeline	5
6.3 Example format	5
6.4 Categories and statistics	5
6.5 Language rationale	6
6.6 Data leakage / hold-out	6
7. Training Configuration	6
8. Export and Deployment	6
8.1 Adapter → GGUF	6
8.2 Registering in Ollama (keeping both models)	7
9. Evaluation / Comparison Methodology	7
9.1 Model switching with no code change	7
9.2 Metrics	7
9.3 Datasets used for comparison	7

9.4 Results (20-sample A/B run)	7
10. End-to-End Workflow	8
11. Reproducibility	9
12. Limitations and Future Work	9
Appendix A — Glossary	9

1. Executive Summary

This report documents the complete process of fine-tuning the **local answer-generation model** of the Cyber-Case Intelligence Framework into a **MITRE ATT&CK cybersecurity specialist**, while **preserving the original model** so the two can be compared head-to-head (A/B).

The base model qwen2.5:7b (served locally through Ollama) is adapted with **QLoRA** using a domain dataset generated directly from the **MITRE ATT&CK STIX 2.1** knowledge base. Training runs on a free cloud GPU (Google Colab / Kaggle, Tesla T4 16 GB) because the development workstation's GPU (NVIDIA RTX 2050, 4 GB VRAM) is too small to fine-tune a 7B model. The trained LoRA adapter is merged and exported to **GGUF (Q4_K_M)** and registered in Ollama as a **separate** model, mitre-qwen:7b. Because the RAG pipeline selects its local model purely through the LOCAL_LLM_MODEL environment variable, the two models can be swapped for evaluation **without changing any pipeline code**.

Item	Value
Base model	Qwen/Qwen2.5-7B-Instruct (Ollama: qwen2.5:7b)
Method	QLoRA (4-bit NF4) + LoRA adapter
Framework	Unsloth + TRL SFTTrainer + PEFT
Training data	MITRE ATT&CK STIX 2.1 (Enterprise + Mobile, v19.0)
Dataset size	3,301 examples (3,037 train / 264 validation)
Output language	English (translation to Thai is a separate pipeline stage)
Training compute	Cloud GPU — Tesla T4 16 GB (Colab/Kaggle)
Deployment	GGUF Q4_K_M → Ollama model mitre-qwen:7b
Comparison	A/B via LOCAL_LLM_MODEL swap, no pipeline change

2. Objectives and Constraints

2.1 Objectives

1. Specialise the local generation model in MITRE ATT&CK knowledge (techniques, tactics, mitigations, groups, software, campaigns).
2. **Keep the original model intact** to enable a fair quality comparison.
3. Integrate the specialist into the existing RAG pipeline with **zero code changes** to the pipeline itself.

4. Make the whole process **reproducible** and runnable on free infrastructure.

2.2 Hardware constraint (the key design driver)

The development workstation has an **NVIDIA RTX 2050 with only 4 GB of VRAM**. QLoRA fine-tuning of a 7B model needs roughly 8–16 GB, so **local training is infeasible**. The resolution adopted throughout this work:

- **Train** on a free cloud GPU (Tesla T4, 16 GB).
 - **Export** the result to a quantised GGUF file.
 - **Run inference** locally through Ollama, where a 7B Q4 model fits comfortably.
-

3. Base Model

The framework already uses qwen2.5:7b via Ollama as the `reasoning_llm` (the component that generates the final answer) whenever the pipeline runs in `--local` mode. To keep the comparison *apples-to-apples*, fine-tuning starts from the matching Hugging Face checkpoint **Qwen/Qwen2.5-7B-Instruct** (same family, size, and chat template as the Ollama base).

Why Qwen2.5-7B-Instruct:

- Strong multilingual capability (relevant for the Thai-facing framework).
 - 7B fits a free T4 in 4-bit and runs locally in Ollama after quantisation.
 - ChatML template is well supported by Unsloth and Ollama.
-

4. Fine-Tuning Method

4.1 LoRA (Low-Rank Adaptation)

Instead of updating all ~7.6 billion parameters, **LoRA freezes the base model** and injects small trainable low-rank matrices into selected linear layers. Only these adapter matrices (tens of MB) are trained. Benefits:

- Fits small GPUs and trains quickly.
- Produces a tiny, portable artefact (the *adapter*).
- The base model is untouched, which aligns directly with the “keep the original model” objective.

4.2 QLoRA (Quantised LoRA)

QLoRA loads the frozen base model in **4-bit (NF4)** precision, then trains the LoRA adapter on top in higher precision. This roughly halves the memory of the base weights, which is what allows a 7B model to be fine-tuned on a 16 GB T4.

4.3 LoRA configuration

Hyper-parameter	Value
Rank (r)	16
lora_alpha	16
lora_dropout	0.0
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Bias	none
Gradient checkpointing	Unsloth (memory-efficient)
Quantisation	4-bit NF4 (load_in_4bit=True)

4.4 Response-only loss masking

Each training example is a chat conversation. Loss is computed **only on the assistant tokens** (the question/context is masked out) using Unsloth's `train_on_responses_only` with the Qwen2.5 ChatML markers (`<|im_start|>user\n / <|im_start|>assistant\n`). This teaches the model *how to answer* rather than to reproduce the prompt, improving answer quality.

5. Tooling / Frameworks

Library	Role
Unsloth	Fast, low-VRAM 4-bit QLoRA for Qwen2.5; model load, LoRA, GGUF export
TRL SFTTrainer	Supervised fine-tuning loop
PEFT	LoRA adapter implementation
bitsandbytes	4-bit quantisation / 8-bit optimiser
datasets	Loads the JSONL training data
llama.cpp	(fallback) HF → GGUF conversion and quantisation
Ollama	Local serving of both the original and the fine-tuned model

Observed cloud environment during a training run: *Unsloth 2026.6.1, Transformers 5.5.0, Torch 2.11.0+cu128, CUDA 12.8, Tesla T4 (16 GB), fp16.*

6. Dataset

6.1 Source

The dataset is generated **directly from the MITRE ATT&CK STIX 2.1 bundles** (no external LLM teacher / API). The builder reuses the framework's existing `StixParser` (`ingestion/stix_parser.py`) to parse the **latest** bundle of each domain (Enterprise and Mobile, **version 19.0**) into typed entities and relationships.

Note: the project's `config.ATTACK_DOMAINS` points at a path that does not exist on disk; the builder resolves the real STIX location (the repository root `Mitre_ATT&CK Doc/`) itself.

6.2 Construction pipeline

1. Parse the latest Enterprise + Mobile STIX bundles → entities + relationships.
2. Build relationship indices (technique→mitigations, group→techniques, software→techniques, tactic→techniques, campaign→group, etc.).
3. Clean MITRE markdown noise (citations, hyperlinks) and truncate long descriptions on sentence boundaries.
4. Emit chat-format examples per template, deduplicate by question, optionally cap per category, shuffle (seeded), and split into train/validation.

6.3 Example format

Each line of the `.jsonl` file is one example with a `messages` array (system / user / assistant), the standard chat format for instruction tuning:

```
{"messages": [
  {"role": "system",    "content": "You are a MITRE ATT&CK cybersecurity specialist..."},
  {"role": "user",      "content": "What mitigations exist for Systemd Timers (T1053.006)?"},
  {"role": "assistant", "content": "Systemd Timers (T1053.006) is a MITRE ATT&CK technique - ..
], "category": "mitigation_lookup", "style": "closed", "language": "en"}
```

Two complementary styles are mixed:

- **closed-book** (3,159 examples) — direct question → answer; bakes MITRE knowledge into the weights.
- **grounded / RAG-style** (142 examples) — a *retrieved context* block precedes the question, training the model to answer **from the provided context only**, matching how the pipeline calls the model at inference time.

Answer wording deliberately mirrors the gold `reference_answer` style used by the framework's evaluation datasets, so the comparison metrics (faithfulness, correctness, ROUGE) measure the right target distribution.

6.4 Categories and statistics

Category	Count	Category	Count
technique_lookup	600	software_type_query	600
mitigation_lookup	600	technique_groups	515
software_techniques	600	group_techniques	173
group_software	161	tactic_techniques	27
campaign_attribution	25	Total	3,301

Split: **3,037 training / 264 validation** (~8% validation, seed 42). Built with `--max-per-category 600` to balance the otherwise software-heavy distribution.

The `technique_detection` category is empty because the current STIX export contains no detects relationships.

6.5 Language rationale

All training data is **English**. In the pipeline the reasoning_llm is explicitly English-in / English-out; translation to Thai is performed by a **separate downstream stage** (`cross_lingual.py`). Training the specialist in English therefore matches its actual role and avoids conflicting with the translation stage.

6.6 Data leakage / hold-out

The evaluation datasets enumerate nearly the entire knowledge base (e.g. `eval_dataset.json`'s 136 rows reference ~1,700 STIX IDs, including ~950 software). A strict entity-level hold-out therefore removes whole entity classes and defeats the goal of building a specialist. The builder defaults to `--holdout none` (train on the full KB) and offers `--holdout ids` as an opt-in for a leak-reduced generalisation test. Accordingly, the default A/B result is best interpreted as **in-domain knowledge absorption** (training and evaluation use different question phrasings).

7. Training Configuration

Setting	Value
Base	Qwen/Qwen2.5-7B-Instruct, 4-bit
Epochs	2
Learning rate	2e-4
LR scheduler	cosine, warmup ratio 0.03
Per-device batch size	2
Gradient accumulation	8 (effective batch = 16)
Max sequence length	2,048
Optimiser	adamw_8bit
Weight decay	0.01
Precision	fp16 (T4 has no bf16)
Seed	42
Loss	assistant tokens only (masked prompt)

A quick `--max-steps 5` “smoke test” is run first to validate the full loop before committing to the complete run.

8. Export and Deployment

8.1 Adapter → GGUF

After training, the LoRA adapter is merged into the base model and converted to **GGUF** quantised to **Q4_K_M**. Two paths are supported:

- **Unsloth direct export** — `model.save_pretrained_gguf(..., "q4_k_m")` (single step, used by the Colab notebook with `--gguf`).

- **Manual path** — `merge_and_gguf.py` merges via Transformers/PEFT, then converts/quantises with `llama.cpp` (`convert_hf_to_gguf.py` + `llama-quantize`).

8.2 Registering in Ollama (keeping both models)

An Ollama Modelfile points at the `.gguf`, sets the Qwen2.5 ChatML template and a MITRE specialist system prompt, then:

```
ollama create mitre-qwen:7b -f export/Modelfile
ollama list    # shows BOTH qwen2.5:7b (original) and mitre-qwen:7b (fine-tuned)
```

The original `qwen2.5:7b` is never modified — the specialist is a **separate** Ollama model.

9. Evaluation / Comparison Methodology

9.1 Model switching with no code change

The pipeline (`agent_graph.py`, `chain.py`, etc.) reads the local model name from `LOCAL_LLM_MODEL` in `config.py`. The comparison runner therefore launches the **existing** generation evaluation (`evaluation/eval_runner.py`) twice as subprocesses, setting `LOCAL_LLM_MODEL` to each model in turn. A fresh process per model avoids the config being import-cached, and **no pipeline or evaluation code is modified**.

9.2 Metrics

The existing generation evaluation reports:

- **Faithfulness (RAGAS)** — is the answer grounded in retrieved context?
- **Answer Relevancy (RAGAS)** — is the answer on-topic?
- **Answer Correctness (RAGAS)** — agreement with the reference answer.
- **Token F1 / ROUGE-L** — lexical overlap with the reference.
- **BERTScore F1** — semantic similarity to the reference.
- **Average latency (ms)**.

The runner parses both reports and renders a side-by-side table with a delta column (per-metric improvement of the fine-tuned model over the base).

9.3 Datasets used for comparison

- `eval_dataset.json` — curated set, the cleaner comparison target.
- `Thai_dataset.json` — large in-domain benchmark (note KB overlap, Section 6.6).

Prerequisites: Neo4j + Qdrant running and ingested, and Ollama serving both models.

9.4 Results (20-sample A/B run)

The comparison was run on 20 samples from `eval_dataset.json` with the RAGAS judge set to **Claude Haiku** (`claude-haiku-4-5`) and RAGAS embeddings served locally by **Ollama `nomic-embed-text`** (so the

LLM-judged metrics no longer depend on a cloud embedding key, and judge concurrency was throttled to stay under provider rate limits). Higher is better for every metric except latency.

Metric	qwen2.5:7b (base)	mitre-qwen:7b (fine-tuned)	Change (ft vs base)
Faithfulness (RAGAS)	0.243	0.610	+0.367
Answer Correctness (RAGAS)	0.351	0.305	-0.046
Token F1	0.131	0.226	+0.095
ROUGE-L	0.082	0.150	+0.068
BERTScore F1	0.826	0.857	+0.031
Avg Latency (ms)	40,765	31,657	-9,108 (faster)

The fine-tuned specialist wins on five of six metrics. The standout is **Faithfulness (+151% relative, 0.243 -> 0.610)**: the specialist grounds its answers in the retrieved MITRE context far more tightly and hallucinates much less — the most important property for the prosecutor use-case, where fabricated technique/ID claims are unacceptable. It is also ~22% faster, because its answers are more concise (it adopts the terse, on-format MITRE answer style baked in during training).

The single regression is **Answer Correctness (-0.046)**. RAGAS answer-correctness rewards covering the reference answer’s facts; the specialist’s terser replies omit some of them, trading completeness for groundedness. This matches the qualitative behaviour observed during testing (it answers the in-distribution part of a question precisely but stops short on compound, multi-part prompts). Adding incident-style, multi-part training examples (Section 12) is the natural way to recover it.

Scope: a 20-sample, in-domain run (dataset built with `--holdout none`), so it measures knowledge absorption rather than out-of-distribution generalisation. The deterministic metrics (Token F1, ROUGE-L, BERTScore) and the RAGAS metrics agree on the overall direction.

10. End-to-End Workflow

- [1] Build dataset (dev box)


```
python data/build_dataset.py --max-per-category 600
-> data/output/train.jsonl, val.jsonl
```
- [2] Train (Cloud GPU – Colab/Kaggle T4)


```
open train/Finetune_Colab.ipynb (or: python train/train_unsloth.py --gguf)
-> LoRA adapter (+ GGUF Q4_K_M)
```
- [3] Export + register (dev box)


```
ollama create mitre-qwen:7b -f export/Modelfile
-> Ollama now has qwen2.5:7b AND mitre-qwen:7b
```
- [4] Compare (dev box, DBs up)

```
python compare/run_comparison.py --max-samples 20
-> compare/comparison_<dataset>.md (side-by-side metrics)
```

11. Reproducibility

- **Seed** 42 used for dataset shuffling/splitting and training.
- All hyper-parameters are centralised in `ft_config.py`.
- Dataset is regenerable from versioned STIX bundles (Enterprise/Mobile v19.0).
- The Colab notebook pins the install and clones the exact branch (`feat/finetune-mitre-specialist`).
- Module file map:

Path	Purpose
<code>ft_config.py</code>	Central configuration (models, paths, hyper-parameters)
<code>data/build_dataset.py</code>	STIX → JSONL builder (reuses <code>StixParser</code>)
<code>data/templates.py</code>	Per-category Q&A templates
<code>train/train_unsloth.py</code>	QLoRA trainer
<code>train/Finetune_Colab.ipynb</code>	One-click Colab training notebook
<code>export/merge_and_gguf.py</code>	Adapter merge + GGUF conversion (<code>llama.cpp</code> path)
<code>export/Modelfile</code>	Ollama model definition for <code>mitre-qwen:7b</code>
<code>compare/run_comparison.py</code>	A/B evaluation runner

12. Limitations and Future Work

- **Template-generated answers** are factually grounded but stylistically uniform. A future iteration could distil richer, incident-style answers from a stronger teacher model (e.g. Claude) for the four-section incident format used by the pipeline's `REASONING_SYSTEM_PROMPT`.
 - **No technique_detection** examples because the current STIX export lacks detects edges; adding detection data sources would broaden coverage.
 - **Default in-domain evaluation** — for a strict generalisation claim, rebuild with `--holdout ids` and report that number separately.
 - **Single quantisation** (`Q4_K_M`) is exported; higher-fidelity variants (`Q5_K_M` / `Q8_0`) could be compared for the quality/size trade-off.
 - A LoRA over the instruct base preserves general ability, so the specialist also serves the pipeline's translation/routing roles in `--local` mode; a dedicated evaluation of those secondary roles is left as future work.
-

Appendix A — Glossary

- **LoRA** — Low-Rank Adaptation; trains small adapter matrices on a frozen model.

- **QLoRA** — LoRA on a 4-bit quantised base model.
- **Adapter** — the small trained weights produced by LoRA.
- **GGUF** — the file format used by llama.cpp / Ollama for quantised models.
- **Q4_K_M** — a 4-bit k-quant GGUF variant balancing size and quality.
- **ChatML** — the chat template format used by Qwen2.5 (<|im_start|> markers).
- **RAGAS** — a library of LLM-based RAG evaluation metrics.